

IN3140/IN4140 - Assignment 4

Due: Friday, May 19th 2023, 23:59 (24h)

Introduction

This assignment has five tasks. In Tasks 1-4 we will look more closely at control of a robot. We will continue to work with the CrustCrawler robot, and by the end of this assignment you will get to test your controller in a simulation environment modeling the real robot.

We will design controllers for a single joint of the simplified three-link Crust-crawler robot. Since Joint 2 does most of the work, it has the most important controller to tune and will therefore be the one we are working with in this assignment. We can imagine the link of the CrustCrawler robot as an inverted pendulum (Figure 2).

For this assignment you will need to use the Red Hat Enterprise Linux (RHEL) clients at IFI with our container-based ROS setup. The simulation environment and control framework are split across two ROS versions: the simulator runs in ROS1, while the controller is implemented in ROS2. Communication between these systems is handled through a dedicated ROS bridge. All required containers are pre-configured, but you may need to download additional assignment-specific packages. You may also use your own computer; however, support for installation and setup will not be provided. At the end of this assignment, you will have relevant experience for working with robotic systems in the real world.

Task 1: Control Theory (40%)

We begin where we left off in **Task 2 (Dynamics II) of Assignment 3**. There, you derived an equation of motion for Joint 2. We shall now work further with this equation.

- a) 10%** Make the equation of motion for Joint 2 independent (of other joints). Justify your method.

HINT Besides figuring out which variables are constant, remember from chapter 7.3 that the quadratic terms represent the effect induced on Joint i by other joints.

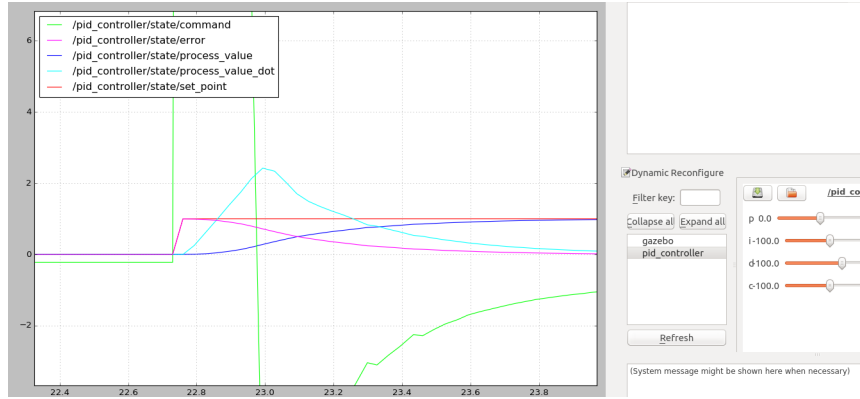


Figure 1: Control of CrustCrawler Joint

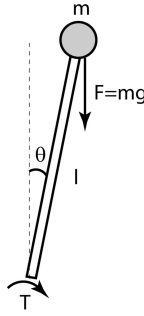


Figure 2: Inverted pendulum model

If you are unable to solve this task for some reason related to Assignment 3, you are allowed to use the generic inverted pendulum torque equation in place of τ_2 for all later tasks (by doing so, you will get **no points** for this subtask and make controller tuning more difficult for yourself):

$$\tau_{pendulum} = ml^2\ddot{\theta} - mgl \sin(\theta) \quad (1)$$

where $m = m_2 + m_3$ and $l = L_2 + L_3$.

- b) 10% Transform the following independent joint control equation from time domain to Laplace domain:

$$u(t) = J\ddot{\theta}(t) + B\dot{\theta}(t) + D(t) \quad (2)$$

Next, write out the terms in the equation you obtained in (a) that correspond to the terms J, B and D in the above equation.

- c) 10% Draw a closed-loop block diagram for equation (2) in Laplace domain, using only simple blocks. Add a PD-controller to the block diagram and derive the transfer function between the input $\theta_d(s)$ and the output $\theta(s)$.
- d) 10% With the PD-controller, the closed-loop system is now second order, and hence the step response is given by the closed-loop natural frequency

ω and damping ratio ζ . Given the requirements of a natural frequency of 6 and a critically damped system, find values for K_P and K_D .

HINT See Chapter 6.3 (which you should read thoroughly) in the textbook.

Next, we will implement the PD-controller and test the values for K_D and K_P that you just calculated. A bit of preparation is in order before we can begin.

Task 2: Setup (5%)

To start the required environments, log in to a RHEL desktop client at IFI and run the following commands in **three separate terminals**:

```
$ in3140_ros1
$ in3140_ros_bridge
$ in3140_ros2
```

The three commands above start three separate environments: terminal #1 (in3140_ros1) runs the ROS1 simulator, terminal #2 (in3140_ros_bridge) runs the ROS1–ROS2 bridge, and terminal #3 (in3140_ros2) runs the ROS2 controller. The sections below correspond directly to these terminals. Please follow the setup in this order.

1) Terminal #1 – CrustCrawler Gazebo Simulation (ROS1)

In terminal #1, set up the ROS1 workspace:

```
$ mkdir -p crust_crawler_simulator/src
$ cd crust_crawler_simulator/src
$ git clone https://github.uio.no/IN3140/crustcrawler_simulation.git
$ git clone https://github.uio.no/IN3140/crustcrawler_pen.git
$ cd ../
$ source /opt/ros/noetic/setup.bash
$ catkin_make
$ source devel/setup.bash
```

Then launch the simulator:

```
$ roslaunch crustcrawler_pen_gazebo controller.launch
```

This will start the CrustCrawler robot in the Gazebo simulation environment.

2) Terminal #2 – ROS1–ROS2 Bridge

In terminal #2, set up and start the bridge:

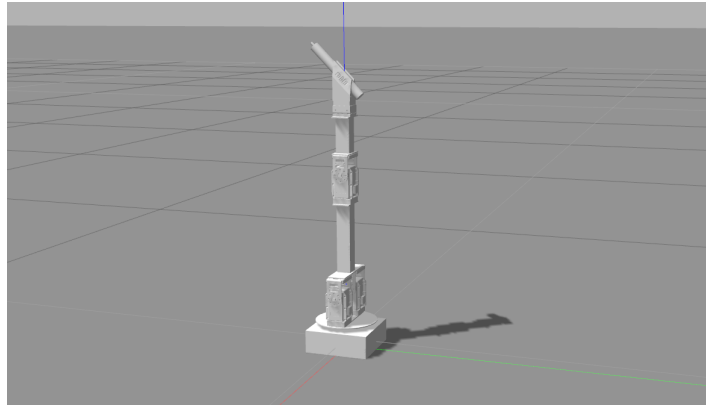


Figure 3: CrustCrawler robot in simulation environment

```
$ source /opt/ros/foxy/setup.bash
$ source install/setup.bash          % ROS_distro warnings are normal
$ source /opt/ros/noetic/setup.bash % ROS_distro warnings are normal
$ rosparam load bridge.yaml
$ ros2 run ros1_bridge parameter_bridge
```

This enables communication between the ROS1 simulator (terminal #1) and the ROS2 controller (terminal #3).

3) Terminal #3 – ROS2 PID Controller

Before starting terminal #3, download the **pid_assignment.zip** file from the course webpage (Mandatory Assignments section).

To run the commands below, make sure you are inside the **in3140_ros2** container and in a directory where the **pid_assignment.zip** file is located. This can be done either by navigating to the correct folder inside the container or by copying the zip file to the default working directory (typically your home directory).

You can verify your current location using the **pwd** command.

In terminal #3, run:

```
$ unzip pid_assignment.zip
$ cd pid_assignment
$ source /opt/ros/humble/setup.bash
$ colcon build --symlink-install
$ source install/setup.bash
```

Then launch the controller:

```
$ ros2 launch pid_assignment pid.launch.py
```

This should open an **rqt** window where the controller can be tuned. If you try to change values now, nothing will happen because the controller is not yet implemented.

Task 3: Simulation I (35%)

We will now work with controlling the position of Joint 2. In the file ‘pid.py’ (located in the src folder) you will find the skeleton implementation which is called by the simulation.

- a) 15% Implement the PD-controller. The return value from the function ‘`__call__(...)`’ should be your computed control effort. Plot the model with the PD-controller. Did the response of the model have a settling time as expected? Why or why not? Include an image of the plot.

HINT *The settling time is the time the response takes to settle within 5% of the steady state value, and is given by the following equation*

$$t_s = \frac{4}{\zeta\omega} \quad (3)$$

- b) 10% Tune your controller so that the model has the lowest possible settling time without overshooting. What are the new values of K_P and K_D ? What is the steady state error of the system? Include an image of the plot after tuning.
- c) 5% Decide if the steady state error in task 3 b) is sufficiently small, or if the controller needs further improvement. Justify your answer.
If the steady state error is regarded as too large, use your knowledge of control theory to implement a controller that will adjust for steady state error, tune it and add an image of the plot.
If the steady state error is regarded as small enough then briefly describe what controller you would have implemented if it wasn’t.
- d) 5% Expand the closed-loop block diagram and function you made earlier in Task 1, with regards to Task 3c).

Task 4: Simulation II (10%)

Until now, we have worked with a PD-controller. For many industrial purposes, the improvement of the derivative term comes at a cost. In factories, the process value signal is often noisy, and the derivative of noise leads to instability (remember that the derivative action works on the rate of change of the error). In figure 4 you can see that the derivative action on Joint 6 of the full CrustCrawler robot causes instability. Here, the gray graph is the position of Joint 6, plotted over time.

For processes that do not benefit from the derivative action, a PI-controller is often chosen instead. In fact, it is the most used controller in the industry as well as the focus of this task.

We have already used the dynamics of the system to calculate controller gains, but we are not always so fortunate as to have a model of the system

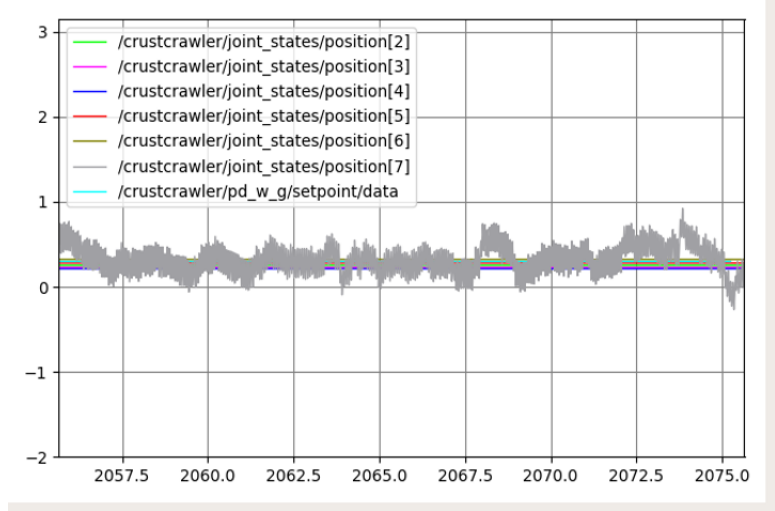


Figure 4: Instability caused by derivative term on Joint 6 of six-link CrustCrawler (with $K_P = 2$ and $K_D = 0.5$)

to compute controller gains from. We will now assume that we know nothing about the system other than that overshoot and oscillations are tolerated.

One popular method that can be used to tune our controller experimentally is called *Ziegler-Nichol's first method*. This method gives us rough estimates of what the different parameters should be. The reason for its popularity is that it can be used without having a model of the system, which is just what we need.

This method uses another form of PID called *standard form* which gives the parameters a clear physical meaning:

$$H(s) = K_p(e(s) + e(s)\frac{1}{T_i s} + e(s)T_d s) \quad (4)$$

where we have that

- K_p : The proportional constant is the *controller gain*.
- T_d : The *derivative time* is the length of time into the future for which the error is estimated and taken into account. In other words, we predict the error value at T_d seconds (or samples) in the future, assuming that the loop control remains unchanged.

The relationship to derivative gain: $K_d = K_p T_d$

- T_i : The *integral time* is the length of time for which past errors have been taken into account. In other words, the integral component adjusts the error value to compensate for the sum of all past errors, with the intention of completely eliminating them in T_i seconds (or samples).

The relationship to integral gain: $K_i = \frac{K_p}{T_i}$

- The equation only holds when $T_d \ll T_i$.

Ziegeler-Nichol's first method can briefly be described as follows:

1. Time parameters are set to; $T_i \approx \infty$ and $T_d = 0$. The regulator is then a P-regulator.
2. The gain K_p is gradually increased, while small disturbances are applied to the process (as steps), until the output signal is a harmonic oscillation (or at least a weakly damped oscillation).
3. We note the value of the proportional constant which gives harmonic oscillations (often also called critical oscillations) as **critical gain**, K_{pk} .
4. The **period** of the harmonic oscillation is noted as

$$T_k = \frac{1}{f_k} = \frac{2\pi}{\omega_{180}} \quad (5)$$

where ω_{180} is the frequency of the harmonic oscillation. In other words, T_k is the time of one period of the harmonic oscillation.

5. Based on the noted values K_{pk} and T_k , the time parameters can be calculated according to table 1 (see appendix), and thus the method works like an autotune for the controller.

- a) 10% Implement a PI-controller in the standard form.

NOTE *The Ziegeler-Nichol's transfer function is designed for continuous signals, while the signal from the simulation actually is discrete. For discretization, multiply T_i by the sampling time Δt . It's not necessary for this assignment; but if you were to implement the derivative action in a discrete system in the future, the discretization will be $(\text{current_error} - \text{previous_error})/\Delta t$.*

- b) 10% Tune the controller using Ziegeler-Nichol's first method.

Task 5: Trajectory Planning Using ROS (10%)

In this final task we will implement a trajectory generator for the CrustCrawler in order to make the robot draw a circle. Everything that is needed to solve this task has been explained in lectures (Chapter 5 - *Path and Trajectory Planning* is not needed). We will first test the code in the Gazebo simulator, and then later, in assignment 5 we can test it on the real robot (assignment 5 is not mandatory, but doing it is highly recommended)! The task is to draw circles, and for this to work, we need to calculate a stream of positions along a trajectory that the robot can follow. The focus of this task will be to create a ROS package and later to see how the same code can work in both simulation and in the real-world.

This task is implemented entirely in ROS1. You do not need ROS2 or the ROS1-ROS2 bridge. Only the `in3140_ros1` container is required. Both the simulator and the trajectory node run inside the `in3140_ros1` container.

You can create a ROS package inside your workspace and implement your trajectory generator there. After building your workspace, you can run your node using `roslaunch`, together with the provided Gazebo simulation.

Below is a markdown file describing the assignment (highlighted in yellow):

Drawing a circle using the CrustCrawler robot

The delivery on this task should be the `path_planner.py` node after completing it.

Appendix I

- Measured masses of links, with motors included.
 - $cm1 = 0.3833 \text{ kg}$
 - $cm2 = 0.2724 \text{ kg}$
 - $cm3 = 0.1406 \text{ kg}$, with pen.
- Inertia tensors

$$\mathbf{I}_1 = \begin{bmatrix} 2.5135 & 0 & 0 \\ 0 & 0.9198 & 0 \\ 0 & 0 & 1.8316 \end{bmatrix} \quad (6)$$

$$\mathbf{I}_2 = \begin{bmatrix} 3.0675 & 0 & 0 \\ 0 & 0.2234 & 0 \\ 0 & 0 & 2.9577 \end{bmatrix} \quad (7)$$

$$\mathbf{I}_3 = \begin{bmatrix} 0.1171 & 0 & 0 \\ 0 & 2.1680 & 0 \\ 0 & 0 & 2.1680 \end{bmatrix} \quad (8)$$

Controller	K_p	T_i	T_d
P	$0.5K_{pk}$	∞	0
PI	$0.45K_{pk}$	$0.85T_k$	0
PID	$0.6K_{pk}$	$0.5T_k$	$0.12T_k$

Table 1: Formulas for Ziegler-Nichol's first method

Appendix II

You can set up the ROS container on your own laptop. First you need to install Apptainer by following these instructions.

Next, you'll need to download the container. To tell Apptainer which library to use, run:

```
$ apptainer remote add --no-login SylabsCloud cloud.sylabs.io
$ apptainer remote use SylabsCloud
```

Now you can download the sif file by running:

```
$ apptainer pull library://vegardds/in3140/in3140_ros:latest
```

To run the container in shell mode, run;

```
$ apptainer shell in3140_ros_latest.sif
```

To add support for nvidia GPU, add the nv flag.

NOTE: There will be no support from faculty in using containers on your own laptops. We encourage you to use IFIs RHEL clients, which we do support.

REQUIREMENTS:

Obtain a total score of at least 40%.

Each student must hand in their own assignment, and you are required to have read the following declaration on student submissions at the department of informatics: <https://www.uio.no/studier/eksamen/obligatoriske-aktiviteter/mn-ifi-obliger-retningslinjer.html>

IMPORTANT! Name the pdf file;

“in3140_oblig4_your_username.pdf”

Submit your assignment at <https://devilry3.ifi.uio.no>.

Your submission must include:

- 1.) A pdf-document with answers to the questions.
- 2.) The code asked for in Task 3 and 4.
- 3.) **A README.txt containing a short reflection on the assignment;** what was difficult, what was easy, was there anything you could have done better?

Where you have used MATLAB, Python or other tools to compute an answer, your approach and solution must be illustrated and explained thoroughly in the pdf file. By illustrating, it's meant that you must paste the block diagrams, the images of the functions you were asked to implement, and the images of the output printed to terminal, ***in the PDF file***.

The files containing the code must also be delivered and named;

“in3140_oblig4_taskXX_your_username.py .m .cpp etc”

You are free to use whatever programming languages and tools you are familiar with, but for this assignment we recommend using Python, as the provided simulator is built using Python. ROS supports only Python and C++ nodes, so you are limited to these two languages for creating ROS nodes.

Deadline: Friday, May 19th 2023, 23:59 (24h)

You can use the mattermost channel *IN3140* for general questions about the assignment and for discussion. mattermost team domain is; <https://mattermost.uio.no/ifi-undervisning/channels/in3140>

Do not hesitate to contact us if you have any further questions.

Artem Chernyshov - artemch@uio.no

Tony Nguyen - hpnguyen@ifi.uio.no

Kristian Roa gran - krisrgra@ifi.uio.no